

# AutoTradeBot

How the app works - an architecture guide for a computer scientist

Repository: [omar3903/ClaudeAutoBot](#) (master, 16 September 2026)

Python 3.10 · FastAPI · SQLAlchemy · ib\_async · pandas/numpy · plain ES-module dashboard

Contents: the twelve books · the big picture · the runtime · a play's life · the strategies · the data and the databases · the research loop and the learning storage · the dashboard and the API · the important functions · a glossary of the trading terms

Companion: [AutoTradeBot-learning.pdf](#), the machine-learning plan in detail

## 1. The twenty books the app was trained on

---

The PDFs live in *OneDrive/Desktop/Training books & Documentation papers/Financial-Economic books/Used books/*. The first four gave the app its trade setups and the way it explains a trade; the next eight gave it the statistics behind the replay, the noise checks, the volatility model, the market regime and the pairs desk. The last eight (added 2026-09-18) are about not fooling yourself: whether a record is luck, what trading really costs, a model that learns which plays pay, and two more books of setups. The last column says where each idea lives in the code.

| Title                                                                                                        | Author            | Edition                                  | What it contributed                                                                                                                                                                                           | Where in the code                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>How to Day Trade for a Living</b>                                                                         | Andrew Aziz       | self-published, 2016                     | the day-trade setups (opening-range breakout, VWAP, flags, ABCD, reversals), 2:1 reward-to-risk, the 2% rule, time-of-day weights, the daily maximum loss, gappers watchlist, scaling out at the first target | strategies/technical.py, analysis/levels.py, risk/position_sizing.py, execution/autopilot.py (daily loss), scanner/heat.py (gappers), execution/exit_manager.py (scale-out) |
| <b>Trading in the Zone</b>                                                                                   | Mark Douglas      | Prentice Hall 2000 (Wiley 2021 printing) | thinking in probabilities: every play states an edge, its odds, and what invalidates it                                                                                                                       | strategies/base.py (the explanation, invalidation and odds fields on every Play)                                                                                            |
| <b>Technical Analysis of the Financial Markets</b>                                                           | John J. Murphy    | New York Institute of Finance, 1999      | horizontal support and resistance, oscillator divergence, trend reading                                                                                                                                       | analysis/levels.py, indicators/ta.py (rsi_divergence, macd_divergence), the divergence_reversal and sr_bounce setups                                                        |
| <b>Financial Modeling and Valuation: A Practical Guide to Investment Banking and Private Equity, 2nd ed.</b> | Paul Pignataro    | Wiley, 2022                              | enterprise value, comparable multiples, discounted cash flow, the football-field band                                                                                                                         | valuation/ (dcf.py, multiples.py, football_field.py, projections.py), strategies/fundamental.py, data/sec_edgar.py (the financials)                                         |
| <b>Quantitative Trading: How to Build Your Own Algorithmic Trading Business</b>                              | Ernest P. Chan    | Wiley, 2009                              | backtests must charge costs, keep a held-out sample, and paper-trade before trusting; half-Kelly sizing                                                                                                       | research/replay.py (costs, held-out third), quant/sizing.py, research/weights.py (live trades count double)                                                                 |
| <b>Algorithmic Trading: Winning Strategies and Their Rationale</b>                                           | Ernest P. Chan    | Wiley, 2013                              | stationarity tests (ADF, Hurst, variance ratio), half-life of mean reversion, buy-on-gap, post-earnings drift, momentum fails in turbulent markets                                                            | quant/stationarity.py, quant/readings.py, scanner/noise.py (not_trending, not_mean_reverting, turbulent_market), strategies/statistical.py                                  |
| <b>Analysis of Financial Time Series, 3rd ed.</b>                                                            | Ruey S. Tsay      | Wiley, 2010                              | GARCH(1,1) volatility forecasting, the market model                                                                                                                                                           | quant/volatility.py, quant/market_model.py (abnormal moves vs SPY)                                                                                                          |
| <b>Applied Econometric Time Series, 4th ed.</b>                                                              | Walter Enders     | Wiley, 2014                              | the same volatility toolkit (RiskMetrics EWMA) and unit-root testing                                                                                                                                          | quant/volatility.py (ewma_var), quant/stationarity.py                                                                                                                       |
| <b>Time Series Analysis</b>                                                                                  | James D. Hamilton | Princeton University Press, 1994         | chapter 22: Markov switching regimes fitted by EM - the calm/turbulent market pill                                                                                                                            | quant/regime.py, engine/market_regime.py (SPY, 3 years of daily returns)                                                                                                    |

| Title                                                                          | Author                | Edition                       | What it contributed                                                                                                                                                                                                                                       | Where in the code                                                                                                                                          |
|--------------------------------------------------------------------------------|-----------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pairs Trading: Quantitative Methods and Analysis</b>                        | Ganapathy Vidyamurthy | Wiley, 2004                   | spread construction, the nonparametric band that maximises profit per crossing, holding periods                                                                                                                                                           | quant/bands.py, pairs/model.py, pairs/desk.py                                                                                                              |
| <b>Likelihood-Based Inference in Cointegrated Vector Autoregressive Models</b> | Søren Johansen        | Oxford University Press, 1995 | the Johansen trace test used to confirm a pair is cointegrated                                                                                                                                                                                            | quant/cointegration.py (johansen), pairs/finder.py                                                                                                         |
| <b>The Cointegrated VAR Model: Methodology and Applications</b>                | Katarina Juselius     | Oxford University Press, 2007 | how to read the cointegration rank in practice; Engle-Granger as the first pass                                                                                                                                                                           | quant/cointegration.py (engle_granger), pairs/finder.py                                                                                                    |
| <b>Evidence-Based Technical Analysis</b>                                       | David R. Aronson      | Wiley, 2006                   | a rule's record is a hypothesis test: bootstrap the mean against zero, test the best of many rules against the best that luck makes (White's reality check), and detrend first so being long in a rising market is not an edge                            | research/significance.py (luck_test, reality_check), research/replay.py (drift_r on every trade), execution/autopilot.py (proof_p_value in the proof rule) |
| <b>Advances in Financial Machine Learning</b>                                  | Marcos López de Prado | Wiley, 2018                   | meta-labelling (the setups pick the side, a model decides whether to act and how big), sample weights by uniqueness with time decay, purged walk-forward folds with an embargo, out-of-sample feature importance, bet size from the predicted probability | research/model.py (Boosted, sample_weights, importance, bet_size), research/validate.py (the folds)                                                        |
| <b>Machine Learning for Algorithmic Trading, 2nd ed.</b>                       | Stefan Jansen         | Packt, 2020                   | gradient boosting on tabular features with gaps, calibrated probabilities, the information coefficient of a feature, time-series cross-validation                                                                                                         | research/model.py (HistGradientBoosting, isotonic calibration, information_coefficients), scripts/train_model.py                                           |
| <b>Trade Your Way to Financial Freedom, 2nd ed.</b>                            | Van K. Tharp          | McGraw-Hill, 2006             | R-multiples and expectancy (the app's unit of account), the quality of an R-multiple distribution (the number he later named SQN), the marble-bag simulation of drawdowns, the percent-risk sizing model                                                  | research/significance.py (sqn, marble_bag), risk/position_sizing.py                                                                                        |
| <b>Systematic Trading</b>                                                      | Robert Carver         | Harriman House, 2015          | costs are the one number known in advance: a rule that pays more than a third of its pre-cost return in costs trades too fast (the speed limit)                                                                                                           | research/replay.py (cost_r on every trade), research/significance.py (cost_share, SPEED_LIMIT), the proof rule                                             |

| Title                                                                 | Author              | Edition                       | What it contributed                                                                                                                                                                                                | Where in the code                                                                                                                                                                                     |
|-----------------------------------------------------------------------|---------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trading and Exchanges: Market Microstructure for Practitioners</b> | Larry Harris        | Oxford University Press, 2003 | implementation shortfall (the fill against the price at the decision), the spread as the price of immediacy, why a stale limit order is a free option for someone else                                             | engine.py (_chase_check: spread gate, the quote kept at the decision), execution/executor.py (expire_entries), trades.entry_slippage_bps / exit_slippage_bps, research/journal.py (execution_quality) |
| <b>The Art and Science of Technical Analysis</b>                      | Adam H. Grimes      | Wiley, 2012                   | market structure tested statistically; the two templates he trades: the failure test (Wyckoff's spring and upthrust) and the pullback after a momentum thrust, read with a 20-EMA inside 2.25-ATR Keltner channels | strategies/patterns.py (failure_test, trend_pullback)                                                                                                                                                 |
| <b>Encyclopedia of Chart Patterns, 2nd ed.</b>                        | Thomas N. Bulkowski | Wiley, 2005                   | measured statistics for chart patterns: twin bottoms fail 64% of the time until a close confirms them; confirmed, the projected height is met about two times in three and half throw back first                   | strategies/patterns.py (double_bottom: confirmation, measure rule, a stop inside the pattern)                                                                                                         |

Only Murphy and Hamilton are scanned images; the rest are text PDFs. None of the statistics needs SciPy or statsmodels: the models in `quant/` and `research/significance.py` are hand-written NumPy (OLS, Nelder-Mead, EM, the bootstrap). Only the learned model (`research/model.py`) uses scikit-learn, and the app runs without it. `scripts/r/audit_records.R` recomputes the record statistics in base R as an independent check; nothing live ever waits on R.

## 2. The big picture

---

**What it is.** AutoTradeBot is a single-process Python application that watches the US stock market through an Interactive Brokers (IBKR) connection, proposes trades ("plays") from a catalogue of strategies, optionally enters them on its own ("Autopilot"), always manages the exits, and keeps a record of everything in a database. A browser dashboard on `127.0.0.1:8787` shows the state and takes the operator's decisions.

### In one sentence per layer:

- **Dashboard** (`tos_bot/web/`): static HTML plus ES modules; talks REST for actions and listens on a WebSocket for events.
- **Server** (`tos_bot/server/app.py`): a FastAPI app whose routes are thin wrappers over engine methods; `/ws` streams the event bus.
- **Engine** (`tos_bot/engine/`): the conductor. Owns every service, runs seven daemon threads, and is the only writer of the runtime state.
- **Scanner and strategies** (`scanner/`, `strategies/`): turn candles into plays on a schedule.
- **Execution** (`execution/`): Autopilot gates, the Executor sends orders, the ExitManager manages open positions.
- **Research** (`research/`, `quant/`): the replay (backtest), the daily journal, the evidence weights, and the statistical models.
- **Data and brokers** (`data/`, `brokers/`, `persistence/`): market data with an on-disk cache, the IBKR adapter and a simulator, and the SQL layer.

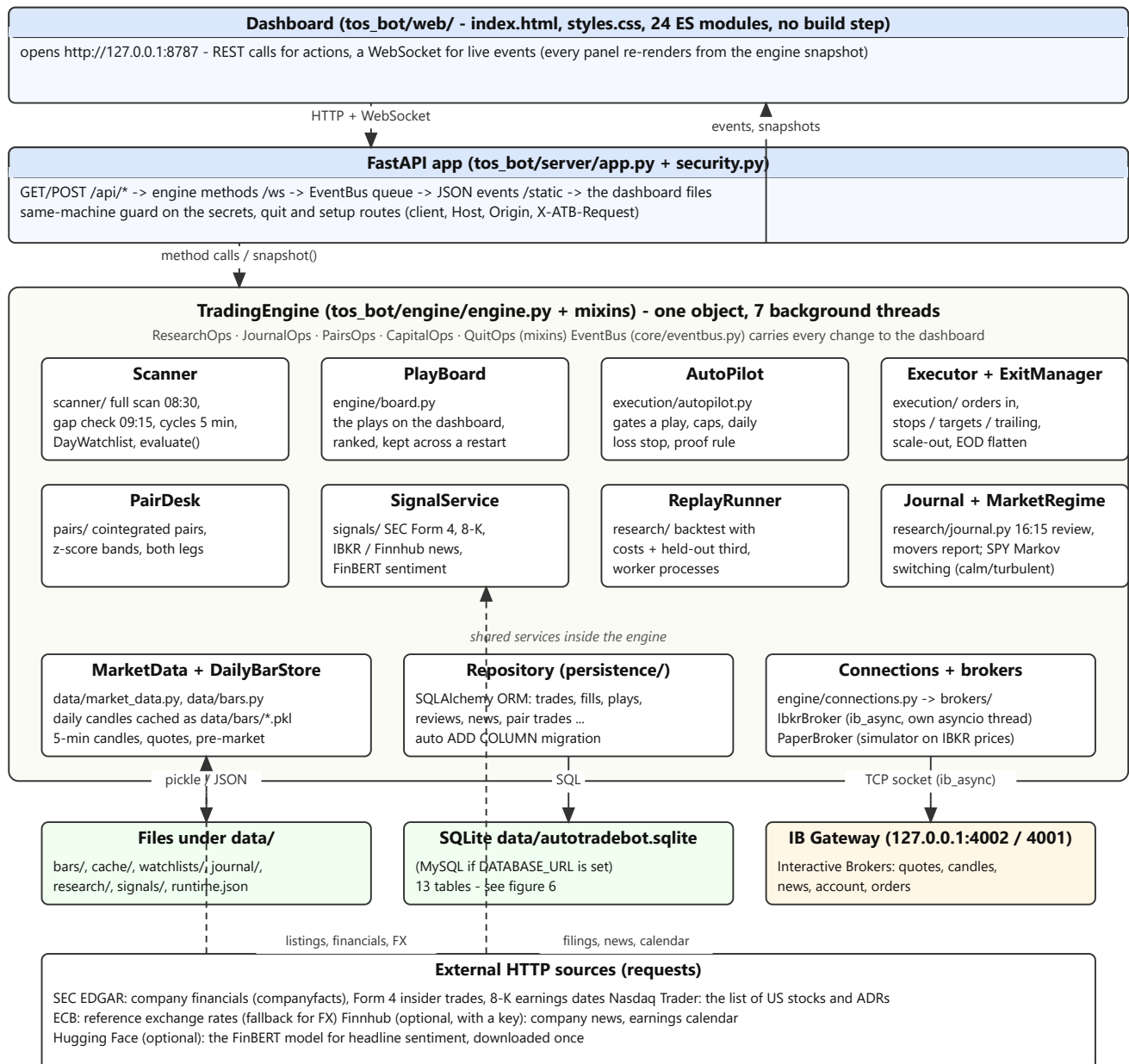


Figure 1 - Components: the browser, the FastAPI server, the engine and its services, and where data comes from and goes to.

## The core classes

Figure 2 is the class view of the same thing. The engine is a composition root: it constructs every service in `__init__`, hands them what they need (the repository, the market data, the event bus) and exposes the operator actions the server calls. The mixin classes only split the engine's source into files by topic; at runtime there is one object. Strategies are stateless: a `StrategyContext` per stock carries the data and the memoised indicators, and `generate()` returns plays.

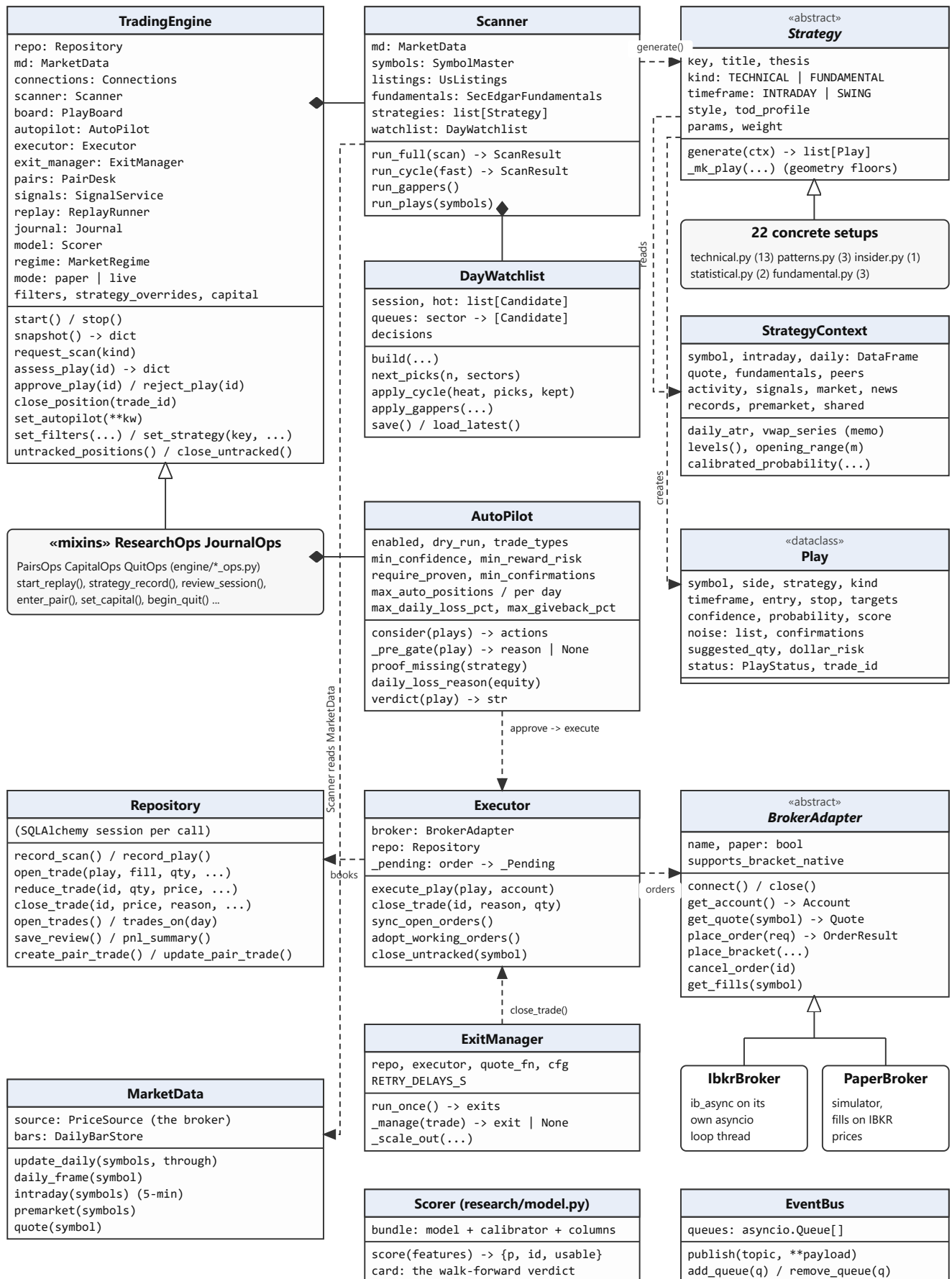


Figure 2 - Class diagram of the core: the engine and what it owns, the scanner and the strategies, the trade path, the broker interface and the persistence layer.

## Technology stack

| Concern            | Choice                                | Notes                                                                                                                                       |
|--------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Language / runtime | Python 3.10, one process              | Started by <code>run.py</code> ; <code>scripts/run_24_7.bat</code> keeps it up                                                              |
| Web server         | FastAPI + uvicorn, WebSocket          | REST routes call the engine in a thread pool; <code>security.py</code> restricts sensitive routes to the same machine                       |
| Broker / prices    | ib_async against IB Gateway           | Paper on port 4002, live on 4001. The only price source; delayed data is accepted                                                           |
| Simulator          | <code>brokers/paper_adapter.py</code> | Fills on IBKR prices; its state is in <code>data/paper_state.json</code>                                                                    |
| Database           | SQLAlchemy 2 ORM; SQLite by default   | MySQL/PyMySQL if <code>DATABASE_URL</code> is set; falls back to SQLite when MySQL is unreachable                                           |
| Numerics           | pandas, numpy                         | No TA-Lib, no SciPy: <code>indicators/ta.py</code> and <code>quant/</code> are vectorised by hand                                           |
| Config             | pydantic-settings                     | <code>.env</code> for secrets, <code>config/config.yaml</code> for defaults, <code>data/runtime.json</code> for dashboard choices (it wins) |
| Dashboard          | Vanilla JS ES modules                 | No bundler, no framework; <code>state.js</code> is a small pub/sub store                                                                    |
| Tests              | pytest, 474 tests in 52 files         | <code>tests/fakes.py</code> fakes the Gateway; <code>-m slow</code> boots the whole engine                                                  |
| Optional ML        | transformers + torch (FinBERT)        | Headline sentiment; the app runs without it                                                                                                 |
| Optional learning  | scikit-learn                          | The meta-label model ( <code>research/model.py</code> ): boosted trees and isotonic calibration; the app runs without it, with no model     |
| Independent audit  | base R (optional)                     | <code>scripts/r/audit_records.R</code> recomputes the record statistics offline; nothing live waits on R                                    |

## How the repository is laid out

|                                 |                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>run.py</code>             | boot the engine + dashboard (Ctrl+C follows the quit rules)                                                    |
| <code>config/config.yaml</code> | defaults for every panel ( <code>config.example.yaml</code> is the template)                                   |
| <code>data/</code>              | everything the app writes (git-ignored): SQLite, candle cache, journal ...                                     |
| <code>tos_bot/</code>           |                                                                                                                |
| <code>config.py</code>          | <code>.env</code> + <code>config.yaml</code> -> Settings (pydantic)                                            |
| <code>core/</code>              | enums, dataclasses (Play, Account, OrderRequest ...), EventBus                                                 |
| <code>engine/</code>            | TradingEngine + mixins, board, connections, reconcile, runtime, regime                                         |
| <code>scanner/</code>           | schedule, heat, watchlist, evaluator, filters, noise, scanner                                                  |
| <code>strategies/</code>        | base (StrategyContext, Strategy), technical, statistical, fundamental,                                         |
| <code>insider</code>            |                                                                                                                |
| <code>execution/</code>         | order_builder, executor, exit_manager, autopilot                                                               |
| <code>risk/</code>              | position_sizing, pdt_guard                                                                                     |
| <code>research/</code>          | replay, runner, history, weights, journal, movers                                                              |
| <code>quant/</code>             | stationarity, cointegration, volatility, regime, bands, sizing,                                                |
| <code>market_model</code>       |                                                                                                                |
| <code>pairs/</code>             | model, finder, backtest, desk                                                                                  |
| <code>signals/</code>           | service, store, book, form4, edgar, news, finnhub, sentiment, calendar                                         |
| <code>data/</code>              | market_data, bars, listings, symbols, sec_edgar, fundamentals, fx, sectors                                     |
| <code>valuation/</code>         | enterprise_value, multiples, dcf, projections, football_field                                                  |
| <code>analysis/</code>          | levels (support/resistance), candles                                                                           |
| <code>indicators/ta.py</code>   | SMA/EMA/RSI/MACD/ATR/Bollinger/Keltner/ADX/VWAP/opening range/divergence                                       |
| <code>brokers/</code>           | base (BrokerAdapter), venues, ibkr_adapter, paper_adapter                                                      |
| <code>persistence/</code>       | db (engine + migration), models_orm (tables), repository (queries)                                             |
| <code>server/</code>            | <code>app.py</code> (routes + WebSocket), <code>security.py</code>                                             |
| <code>web/</code>               | <code>index.html</code> , <code>styles.css</code> , <code>js/*.js</code>                                       |
| <code>util/</code>              | clock (NYSE calendar to 2028), logging, keep_awake, net                                                        |
| <code>tests/</code>             | pytest ( <code>fakes.py</code> = a synthetic Gateway)                                                          |
| <code>scripts/</code>           | <code>init_db.py</code> (MySQL schema), <code>ibkr_setup.py</code> (Gateway doctor), <code>run_24_7.bat</code> |

### 3. The runtime: process, threads and loops

`python run.py` builds the FastAPI app, which constructs one `TradingEngine` and calls `start()`. Start binds the broker chosen by the routing (`brokers/venues.py`: IBKR paper, IBKR live, or the simulator on IBKR prices), reads the account once, and launches the seven daemon threads below. Every thread sleeps on the same `threading.Event`, so `stop()` ends them together. Ctrl+C goes through the quit rules: in paper it closes positions and exits; in live with positions open it asks in the dashboard first.

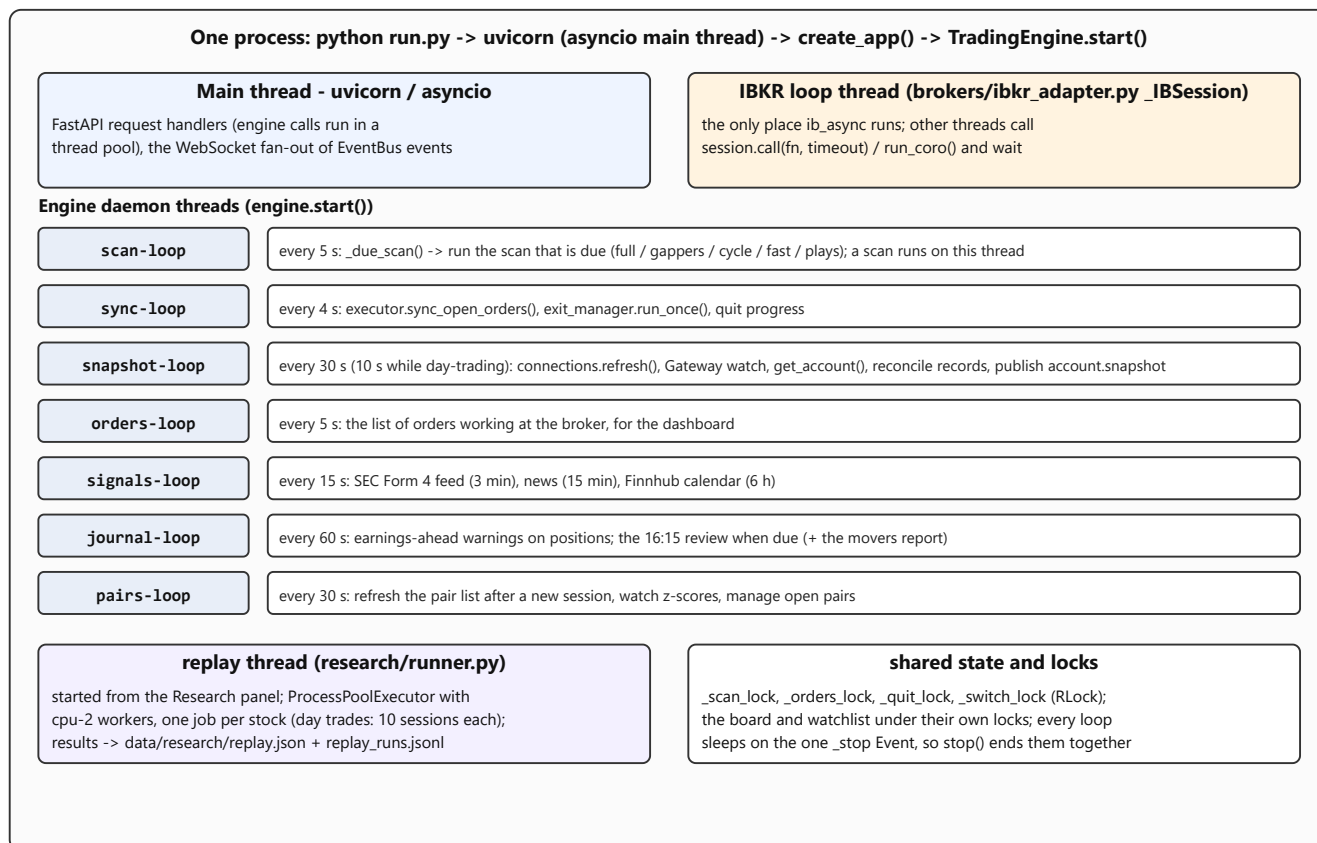


Figure 3 - Threads: everything runs in one Python process; the engine's loops are daemon threads, the replay uses worker processes.

#### The scan schedule (`scanner/schedule.py`)

| Scan kind | When                                                                                  | What it reads                                                                                                                                                                        | What it produces                                                                                                                              |
|-----------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| full      | once a day at 08:30 ET (settable 04:00-09:00); also on start-up if today's is missing | the Nasdaq listing of ~5,700 stocks and ADRs, IBKR contract details, one year of daily candles for every tradable name (top-ups after the first day), SEC financials for the leaders | a ranked universe by daily heat, the day's hot list (20) and per-sector buffers (25 each), swing plays for the top 400 names, valuation plays |
| gappers   | 09:15 ET (settable 08:00-09:25)                                                       | pre-market 5-minute candles of the hot list and buffers                                                                                                                              | swaps the coolest non-gapping hot names for stocks gapping 2%+ on 50k+ shares                                                                 |
| cycle     | every 5 minutes while the market is open                                              | 5-minute candles of the hot list + 2 buffer names per sector                                                                                                                         | day-trade and swing plays; adopt / keep / drop decisions on the buffers                                                                       |
| fast      | every 60 s while Autopilot is day-trading                                             | the hot list only                                                                                                                                                                    | fresh day-trade plays                                                                                                                         |
| plays     | every 15 s                                                                            | the stocks that already have plays on the board                                                                                                                                      | re-evaluates them so stale plays leave                                                                                                        |

The design is deliberate: a full sweep of the market takes about ten minutes of Gateway time (each candle request takes the Gateway about half a second whatever its length; twelve in flight read ~17 stocks/s, and thirty-two time out). So the whole market is read once a day and the day's attention goes to a small, sector-diverse list. Swing setups read daily candles, which change once a day, so rescanning everything intraday would not find more of them.

## 4. The life of a play

A **play** is the app's unit of work: a proposed trade with a symbol, a side (LONG or SHORT), an entry price, a stop (where the idea is wrong), one or two targets, the setup's confidence and odds, noise flags, and a rank score. It is a plain dataclass ( `core/models.py Play` ). Figure 4 follows one from a scan cycle to its exit.

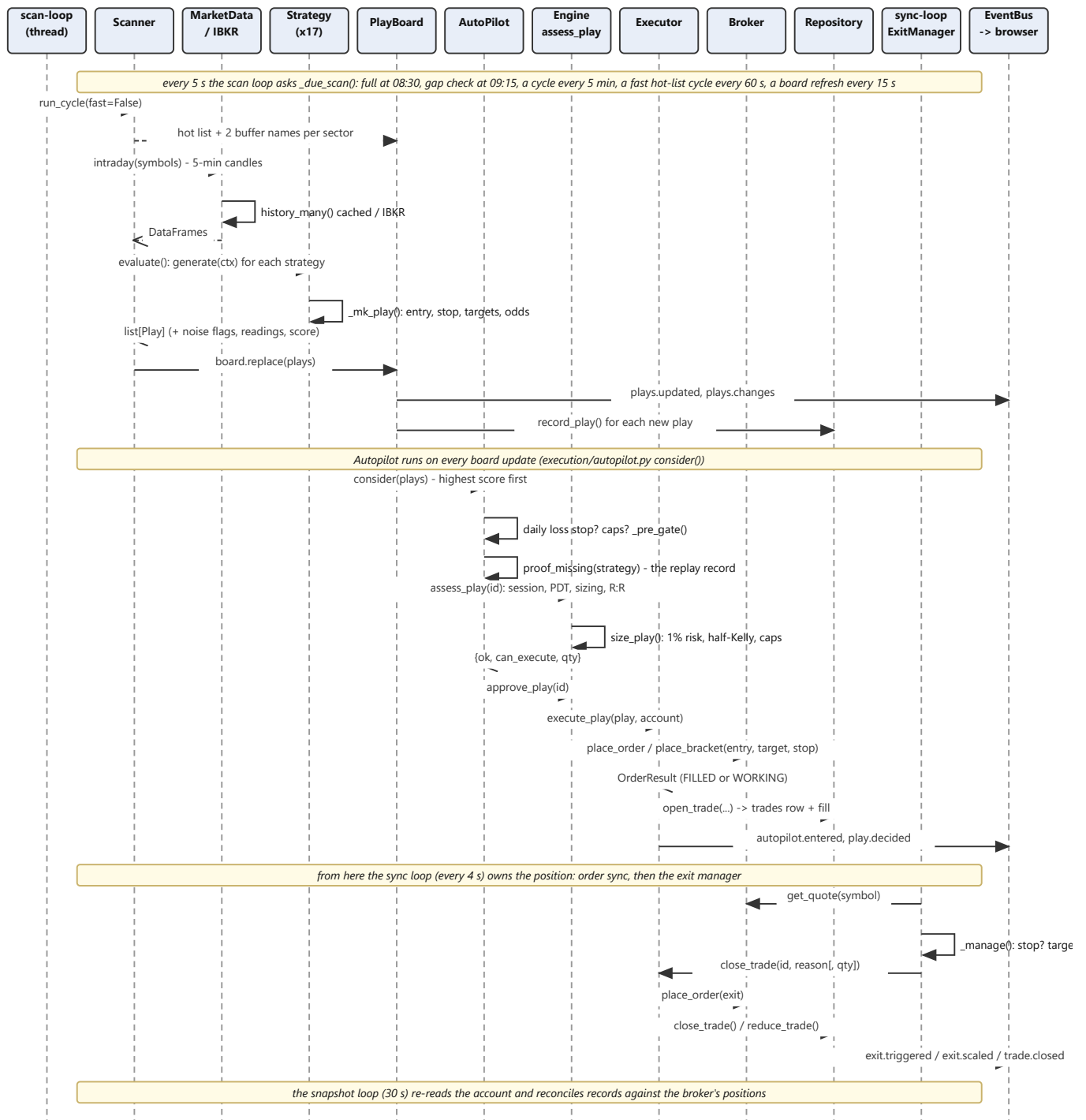


Figure 4 - Sequence: from a scan cycle to an order and, later, to the exit that closes it.

### Step by step

- Candles.** `MarketData.intraday()` returns a pandas DataFrame per symbol (5-minute OHLCV in New York time). Daily candles come from the pickle cache in `data/bars/`, topped up from IBKR.
- Evaluate.** `scanner/evaluator.py evaluate()` builds a `StrategyContext` (candles, quote, activity, signals, market regime, the strategy records) and calls each enabled strategy's `generate(ctx)`. Expensive series

(ATR, VWAP, levels) are memoised on the context so seventeen strategies share one computation.

3. **Geometry guard rails.** `Strategy._mk_play()` refuses plays whose stop is too tight (below 0.6% intraday / 1.5% swing, below 0.9 ATR, or below the GARCH volatility floor) or whose reward-to-risk is outside 0.4-8. A tightened stop cannot fake a good ratio.
4. **Noise and readings.** `scanner/noise.py` flags a play that goes against the trend, against VWAP, into a gap, on a news-driven move, in a turbulent market, or on a non-trending stock (ADF/Hurst). Readings (price character, volatility forecast, next earnings) go into `evidence`.
5. **Score.** `rank_score()` = expected value in R units times the strategy weight, plus small bumps for unusual volume, a gap and enough range.
6. **Board.** `PlayBoard.replace()` keeps the top 80, notes what joined and left, and publishes `plays.updated`. Every new play is logged to `play_logs`.
7. **Autopilot.** `AutoPilot.consider()` walks the board highest score first and applies the gates in the next table. The first failing gate is the reason shown on the dashboard.
8. **Assessment and sizing.** `engine.assess_play()` checks the session, the pattern-day-trader rule (live only), and calls `risk/position_sizing.py size_play()`: risk 1% of equity per trade, lowered by the strategy's half-Kelly (a quarter of the risk until the replay has proven the strategy), the mid-day factor, the open-risk ceiling (4%), the per-position cap (12% of equity) and the per-symbol cap (15%).
9. **Order.** `Executor.execute_play()` builds a limit order (5 bps through the price) and, where the broker supports it, a native bracket with the stop and target attached. An immediate fill opens the trade; otherwise the order is tracked and `sync_open_orders()` books the fill later.
10. **Exit.** The sync loop runs `ExitManager.run_once()` every 4 s on every open trade of this venue (figure 5).
11. **A stop that outlives the app.** On IBKR the executor also keeps a good-till-cancelled stop order resting at the broker for every open trade ( `execution/protective_stops.py` ), made to match the trade record on every pass. It is cancelled, and the cancel confirmed, before the app sends any exit of its own; if it filled first, that fill is booked and nothing else is sent; a stop whose trade is no longer open is swept away. Beside the stop rests a limit order at the target, in one one-cancels-all group with it (IBKR reduces the other by what fills): profit is taken at the target on real prices, even on delayed quotes or with the app off, and the two can never both fill for the whole position.

## The Autopilot gates, in order (execution/autopilot.py)

| Gate                                  | Rule                                                                                                            | Setting                                                                          |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| switched on, venue ok                 | Autopilot on; live trading only when the live checks pass                                                       | enabled                                                                          |
| daily stop                            | no entries once today's closed trades lost 2% of equity, or gave back 30% of the day's peak gain (after +0.25%) | max_daily_loss_pct, max_giveback_pct                                             |
| per-cycle / per-day / concurrent caps | 1 new entry per pass, 5 per day, 10 open, 2 per strategy                                                        | max_new_per_cycle, max_auto_trades_per_day, max_auto_positions, max_per_strategy |
| trade type                            | day and swing follow the filter bar; pairs separately                                                           | trade_types                                                                      |
| confidence                            | the setup's own conviction $\geq$ the floor (0.62 default; PR #22 adds a swing floor of 0.5)                    | min_confidence                                                                   |
| reward-to-risk                        | $\geq 2.0$ (Aziz)                                                                                               | min_reward_risk                                                                  |
| kind                                  | valuation plays are never auto-traded                                                                           | -                                                                                |
| noise                                 | none of the chosen flags, nor the ones the replay taught it to skip                                             | skip_noise + learned                                                             |
| confirmations                         | a day-trade setup must appear in 2 scans in a row                                                               | min_confirmations                                                                |
| catalyst                              | optional: a news or earnings tag                                                                                | require_catalyst                                                                 |

| Gate              | Rule                                                                                                                                                                                                                                                             | Setting                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| already in it     | not holding the symbol, no entry working, not stopped out today (cooldown)                                                                                                                                                                                       | -                                                      |
| late in the day   | no new day trade with fewer than 30 minutes to the close                                                                                                                                                                                                         | min_minutes_to_close                                   |
| <b>proof</b>      | the strategy's replay record: $\geq 30$ trades at $\geq +0.05R$ (also net of the stocks' own drift), a positive held-out sample of $\geq 10$ trades, a reality-check p-value $\leq 0.10$ across every setup tried, and costs within a third of the pre-cost edge | require_proven, proof_p_value                          |
| the learned model | in gate or size mode, and only while its own walk-forward verdict calls it usable: plays it gives under 55% are refused; shadow (the default) only logs its odds                                                                                                 | model_mode, model_min_p                                |
| engine assessment | session valid, PDT ok, size > 0, R:R ok, open-risk and gross-exposure ceilings                                                                                                                                                                                   | risk.*, max_gross_exposure_pct                         |
| the last look     | at the live quote: refused when the spread is over 0.10R of the risk or the price has run 0.25R past the entry; within that the limit is priced off the quote; a day-trade entry unfilled after 10 minutes is cancelled                                          | execution.max_spread_r, max_chase_r, entry_timeout_min |

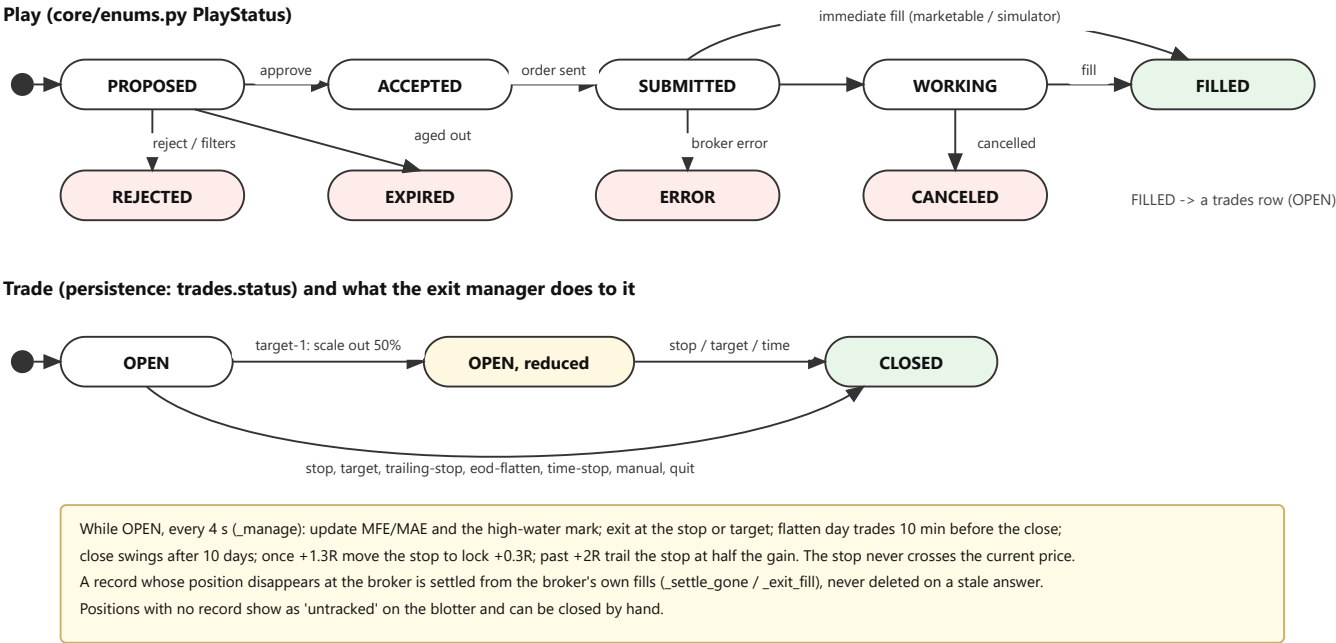


Figure 5 - State machines: a play's status and a trade's life under the exit manager.

## 5. The strategy catalogue

A strategy is a class with a `key`, a `kind` (technical or fundamental), a `timeframe` (INTRADAY = day trade, SWING = multi-day), a `style` (momentum trades with the move, reversal fades it, value ignores price action) and a time-of-day profile that scales confidence by the session hour. Each is switched on and weighted in the Strategies panel; the weight multiplies the rank score, never the geometry. The registry (`strategies/registry.py`) builds them from `config.yaml` plus the dashboard overrides.

| Key                    | Title                            | Timeframe | Style            | Idea in one line                                                                                   | Book         |
|------------------------|----------------------------------|-----------|------------------|----------------------------------------------------------------------------------------------------|--------------|
| opening_range_breakout | Opening-Range Breakout           | day       | momentum         | break of the first N minutes' range on volume                                                      | Aziz         |
| vwap_reclaim           | VWAP Reclaim / Loss              | day       | reversal         | price crosses back over the volume-weighted average price                                          | Aziz         |
| ema_pullback_trend     | Trend Pullback to Moving Average | day       | momentum (trend) | pullback to the 9/20 EMA in a trend                                                                | Aziz         |
| gap_and_go             | Gap & Go                         | day       | momentum         | a gapper that holds its gap and pushes on                                                          | Aziz         |
| abcd_pattern           | ABCD Pattern                     | day       | momentum         | the A-B-C-D continuation                                                                           | Aziz         |
| bull_bear_flag         | Bull / Bear Flag                 | day       | momentum (open)  | a pole and a tight flag in the first hour                                                          | Aziz         |
| red_to_green           | Red-to-Green (prior-day close)   | day       | reversal         | crossing yesterday's close                                                                         | Aziz         |
| intraday_reversal      | Top / Bottom Reversal            | day       | reversal         | an extended move, a reversal candle, entry on the next candle's new high/low                       | Aziz         |
| sr_bounce              | Horizontal Support / Resistance  | day       | reversal         | a bounce off a clustered level                                                                     | Murphy, Aziz |
| rsi2_mean_reversion    | RSI(2) Mean Reversion            | swing     | reversal         | a 2-period RSI extreme in a trend                                                                  | Chan         |
| bollinger_fade         | Bollinger Band Fade              | swing     | reversal         | a close outside the band, fade back to the mean                                                    | Murphy       |
| atr_channel_breakout   | Keltner / ATR Channel Breakout   | swing     | momentum         | a close outside the ATR channel                                                                    | Murphy       |
| week52_breakout        | 52-Week High/Low Momentum        | swing     | momentum         | a new 52-week high on projected volume                                                             | Murphy       |
| divergence_reversal    | Oscillator Divergence at a Level | swing     | reversal         | RSI/MACD divergence at support or resistance                                                       | Murphy       |
| failure_test           | Failure Test (spring / upthrust) | swing     | reversal         | a probe through a swing level that closes back inside; the stop goes just beyond the test          | Grimes       |
| trend_pullback         | Pullback After a Momentum Thrust | swing     | momentum         | the first shallow, quiet pullback to the 20-EMA after a close outside the 2.25-ATR Keltner channel | Grimes       |

| Key                      | Title                          | Timeframe | Style    | Idea in one line                                                                                                  | Book               |
|--------------------------|--------------------------------|-----------|----------|-------------------------------------------------------------------------------------------------------------------|--------------------|
| double_bottom            | Confirmed Double Bottom / Top  | swing     | momentum | twin lows 2-7 weeks apart, taken only on the close beyond the peak between them; the stop sits inside the pattern | Bulkowski, Murphy  |
| gap_reversion            | Gap reversion                  | day       | reversal | buy-on-gap: an overnight gap against the trend reverts                                                            | Chan (Algorithmic) |
| earnings_drift           | Post-earnings drift            | day       | momentum | drift after an 8-K earnings surprise                                                                              | Chan (Algorithmic) |
| relative_value_comps     | Relative Value vs. Peers       | swing     | value    | cheap vs the sector's median multiples                                                                            | Pignataro          |
| dcf_fair_value_gap       | DCF Fair-Value Gap             | swing     | value    | price far below a discounted-cash-flow value                                                                      | Pignataro          |
| valuation_football_field | Football-Field Fair-Value Band | swing     | value    | price outside the band of all methods                                                                             | Pignataro          |
| insider_buying           | Insider buying                 | swing     | value    | clustered open-market buys by officers (Form 4)                                                                   | signals            |

The engine reports 17 technical/statistical/fundamental strategies loaded plus the insider one; valuation plays are shown but never auto-traded.

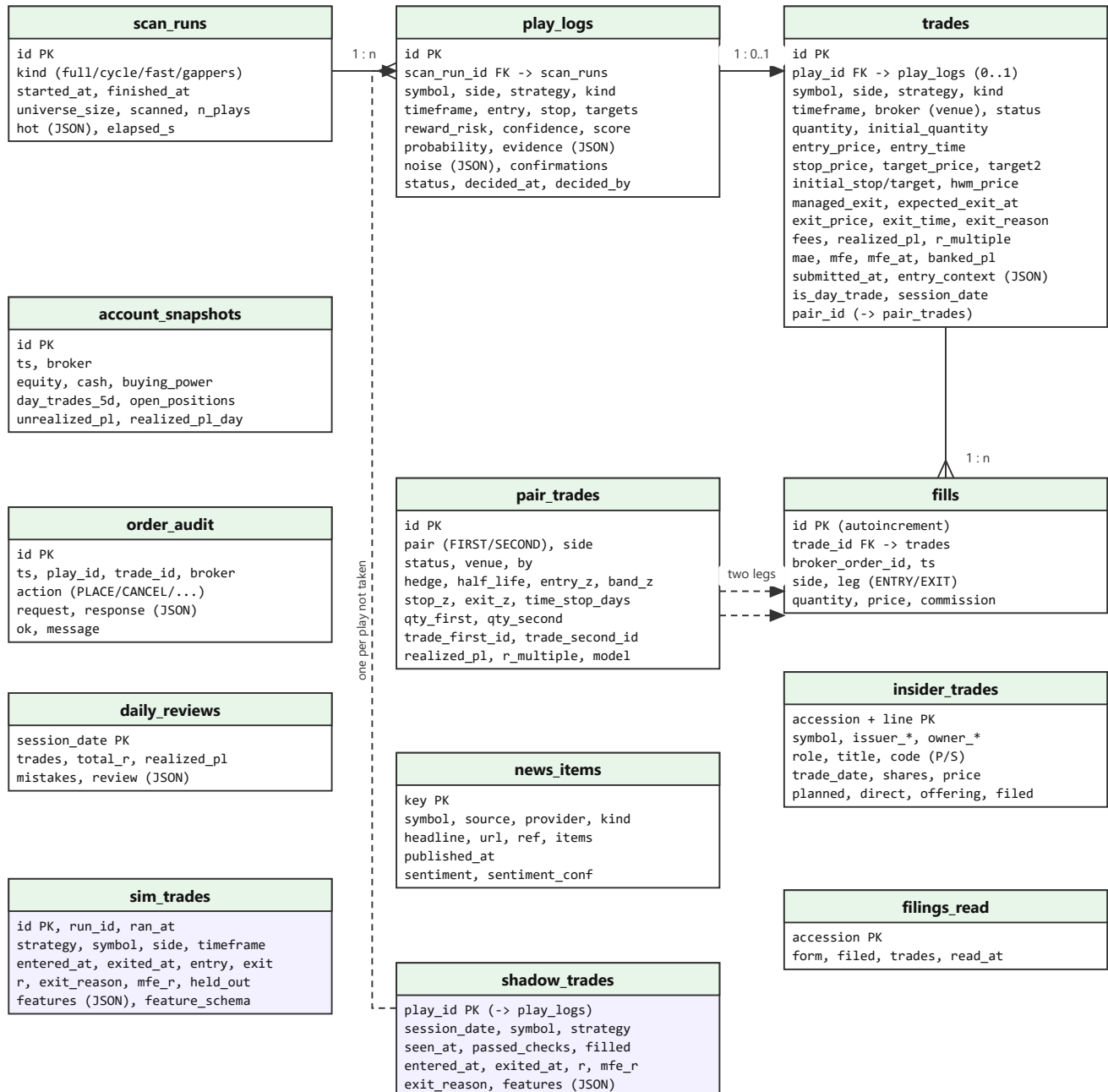
## What every play carries (Play fields you will see in the API and the database)

| Field                                | Meaning                                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| entry, stop, targets                 | prices; the stop is the invalidation, targets are where profit is taken (the first target scales out half if there is a second) |
| reward_risk                          | $(\text{target} - \text{entry}) / (\text{entry} - \text{stop})$ ; Aziz wants $\geq 2$                                           |
| confidence                           | the setup's own conviction, 0-1, scaled by time of day                                                                          |
| probability                          | the odds the play pays, calibrated against the strategy's pooled record                                                         |
| score                                | the rank on the board (expected R times weight plus bumps)                                                                      |
| noise                                | list of flags such as against_trend, news_driven_move, turbulent_market, not_trending                                           |
| confirmations                        | how many scans in a row showed this setup                                                                                       |
| evidence                             | a dict with the numbers behind the play: expected_r, vol_forecast, price_character, market_regime, next_earnings, a sparkline   |
| suggested_qty, dollar_risk, notional | the sizing result                                                                                                               |
| status, trade_id                     | the state machine of figure 6 and the trade it became                                                                           |

## 6. Data: the database, the files and the configuration

### The database

`persistence/db.py` builds one SQLAlchemy engine. With no `DATABASE_URL` in `.env` it tries the MySQL URL built from the `DB_*` settings and, when that is not reachable, falls back to **SQLite at `data/autotradebot.sqlite`** - which is what runs today. `create_all()` creates the tables and `_add_missing_columns()` adds any column the ORM has and the table lacks, so a new field never needs a manual migration. Every `Repository` method opens its own short session ( `session_scope()` ) and returns plain dicts, so the rest of the app never touches ORM objects.



Money columns use a NUMERIC(20,6) type; JSON columns hold dicts/lists. Missing columns are added at start-up (`persistence/db.py _add_missing_columns()`).  
Shaded: what a model learns from (research/dataset.py) - with `trades.entry_context`, the three populations of the learning guide.

Figure 6 - The database (`persistence/models_orm.py`): thirteen tables, SQLite by default.

| Table                         | One row is                                                                                                                                                                                                                                                | Written by                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| scan_runs                     | one scan (full, gappers, cycle, fast, plays) with its timings and counts                                                                                                                                                                                  | engine.run_scan -> repo.record_scan          |
| play_logs                     | one play shown on the board, with its decision (approved, rejected, expired, auto)                                                                                                                                                                        | repo.record_play / set_play_status           |
| trades                        | one position from entry to exit, including partial exits (banked_pl), the R multiple, the play's features at the decision (entry_context) and what the fills cost: decision_price, spread_bps, entry_slippage_bps, exit_decision_price, exit_slippage_bps | repo.open_trade / reduce_trade / close_trade |
| fills                         | each broker fill, entry or exit leg, with commission                                                                                                                                                                                                      | executor via the repository                  |
| account_snapshots             | equity, cash, buying power every snapshot-loop pass                                                                                                                                                                                                       | engine snapshot loop                         |
| order_audit                   | every order request and the broker's answer (JSON)                                                                                                                                                                                                        | executor._audit                              |
| daily_reviews                 | the 16:15 review per session (JSON) plus its headline numbers                                                                                                                                                                                             | JournalOps                                   |
| news_items                    | a headline from IBKR, SEC 8-K or Finnhub, with FinBERT sentiment                                                                                                                                                                                          | SignalStore                                  |
| insider_trades / filings_read | Form 4 open-market trades and which filings were parsed                                                                                                                                                                                                   | SignalStore                                  |
| pair_trades                   | a two-leg pair position with its z-score model and both trade ids                                                                                                                                                                                         | PairDesk                                     |
| sim_trades                    | one simulated trade of a replay run, with the play's features at the signal, a held_out flag, and what the stock's own drift made (drift_r) and costs took (cost_r) while it was held                                                                     | the runner's sink -> repo.save_sim_trades    |
| shadow_trades                 | one play shown and not taken, followed to its outcome on the session's candles, with its features                                                                                                                                                         | the 16:15 review -> repo.save_shadow_trades  |

Three of the columns above are the learning storage: `trades.entry_context` holds the play's features at the fill (with `submitted_at` and `mfe_at`), and the two shaded tables hold the plays not taken and the replay's trades with the same features. `research/features.py` is the one function that produces them, versioned by `FEATURE_SCHEMA`; `research/dataset.py` joins the three into one training table and `scripts/export_training_set.py` writes it as CSV. The learning guide ( `docs/AutoTradeBot-Learning.pdf` ) explains what is done with it.

## Files under data/ (git-ignored, all rebuildable)

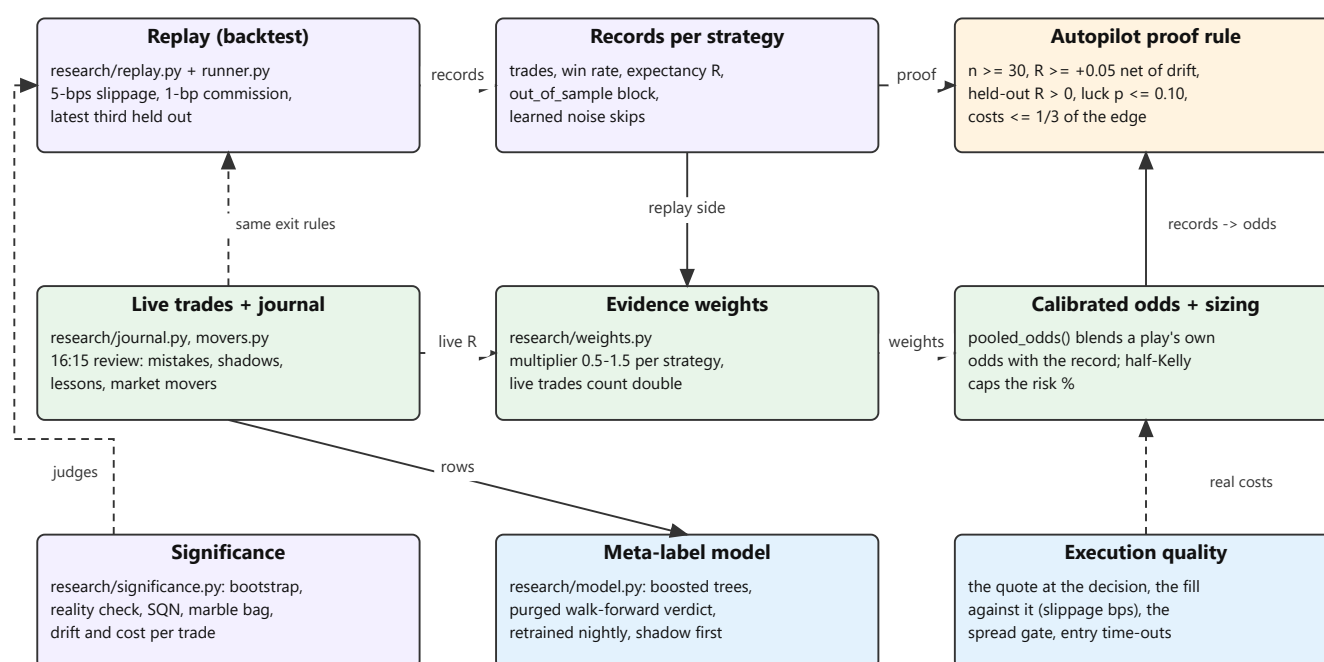
| Path                             | Format                | Holds                                                                                                                                                                                                                 |
|----------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bars/<SYMBOL>.pkl                | pickled DataFrame     | one year of daily candles per stock (5,500+ files)                                                                                                                                                                    |
| research/daily/<SYMBOL>.pkl      | pickled DataFrame     | three years of daily candles (replay.daily_years) for the stocks the replay runs on; one request per stock the first time, then topped up from bars/                                                                  |
| research/models/                 | joblib + current.json | the trained meta-label models and the latest model's card                                                                                                                                                             |
| cache/                           | pickles + JSON        | 5-minute candle windows, the Nasdaq listing, SEC financials (7 days)                                                                                                                                                  |
| symbols.json                     | JSON                  | IBKR contract ids, stock types, sectors (the SymbolMaster)                                                                                                                                                            |
| watchlists/watchlist_<date>.json | JSON                  | the day's hot list, buffers and decisions (5 days kept)                                                                                                                                                               |
| runtime.json                     | JSON                  | every dashboard choice: mode, filters, strategies, capital, scan settings, Autopilot settings - wins over config.yaml                                                                                                 |
| day_state.bin                    | compressed pickle     | the current session so far: the board's plays, the setups settled, the gap check's pre-market levels, the last wide scan's time, the last scans' summaries - a restart picks the day up from it (engine/day_state.py) |
| paper_state.json                 | JSON                  | the simulator's cash, positions and round trips                                                                                                                                                                       |

| Path                                                     | Format  | Holds                                                      |
|----------------------------------------------------------|---------|------------------------------------------------------------|
| research/replay.json,<br>replay_runs.jsonl               | JSON    | the latest replay's records and every run's summary        |
| research/training_set.csv                                | CSV     | the exported training set (scripts/export_training_set.py) |
| research/intraday/                                       | pickles | 5-minute history kept for the replay                       |
| research/benchmark_spy.pkl                               | pickle  | SPY daily returns and the fitted regime model              |
| journal/<date>.json                                      | JSON    | the daily review with the movers report                    |
| signals/state.json,<br>earnings_calendar.json, earnings/ | JSON    | signal service state and earnings dates                    |
| pairs/watch.json                                         | JSON    | the cointegrated pairs being watched                       |

## Configuration precedence

1. `.env` (secrets: IBKR host/ports/client id/account, Finnhub key, DATABASE\_URL). Written only through the Connections panel by `secrets_store.py`, an allow-listed writer.
2. `config/config.yaml`: defaults for every section ( `config.py` AppConfig: account, risk, scanner, strategies, valuation, execution, exit\_manager, autopilot, noise, replay, journal, pairs, signals, database, app).
3. `data/runtime.json`: what you set in the dashboard. Loaded at start and applied over the YAML, so a slider you moved survives a restart and beats the file.

## 7. The research loop: replay, records, journal



Everything on this page is stored under `data/research/` and `data/journal/`, and shown on the Research and Reports panels.

Figure 7 - The evidence loop: the replay proves a setup, live results re-weight it, and both calibrate the odds the plays state.

**Replay** (`research/replay.py`). A backtest that walks candles bar by bar and lets the same strategy code propose plays as if it were live: day trades on 5-minute candles over the last 60 sessions, swings on three years of daily candles (700 sessions; each setup still sees the 300 it would see live), fills at the next bar or next open, 5 bps slippage and 1 bp commission each way, the same exit rules as the exit manager (including scale-out), news flags from the stored headlines, and the market regime as it was known that day. The latest third of sessions is **held out**: a strategy must also work on data it was not tuned on.

**Which stocks, and which way in.** Day-trade setups only ever see the morning's hot list, so each past session is replayed on the stocks the scan would have picked *that morning* (`research/in_play.py`: the 40 hottest by daily heat as of the session before, plus the 10 biggest opening gaps) - not on today's hot list, which is hot because of what it has just done and hands a momentum setup its own hindsight. Each day setup is followed twice: entered on sight (`entry_rule` first - the record of all trades) and entered once it has shown two bars in a row (second - how Autopilot enters when it asks for two scans running); Autopilot's record, the one the proof rule reads, uses the way in it really uses. The replay reads 5-minute candles with flat costs: it is good at throwing out a setup that loses, and a setup it likes still has to show it on paper.

**Runner** (`research/runner.py`). Splits the work into jobs (one stock, or ten sessions of one stock for day trades), runs them on a `ProcessPoolExecutor` with every core but two, and merges the simulated trades into per-strategy records: trade count, win rate, expectancy in R, profit factor, worst drawdown, and the same block for the held-out sample. It also learns which noise checks are worth skipping (`learned_skips`).

**Proof rule.** Autopilot, with `require_proven` on, refuses a strategy whose record does not show at least 30 trades averaging  $+0.05R$  with a positive held-out sample of at least 10. That is why no swing trade was taken on 16 September: the records were negative or too thin. In **Live** the rule is in force whatever the setting says (`AutoPilot.proof_required`); on paper it is the user's choice, so unproven setups can be practised - at a quarter of the usual risk (`research_ops.PRACTICE_RISK`), with the settings in force kept on every trade.

**Evidence weights** ( `research/weights.py` ). Each strategy gets a multiplier between 0.5 and 1.5 from its pooled record (live trades count double), applied to the rank score. `pooled_odds()` blends a play's own stated odds with the record so the probability shown is calibrated, and `quant/sizing.py half_kelly_risk_pct()` can only lower the risk per trade.

**Journal** ( `research/journal.py` , `movers.py` ). At 16:15 the journal loop builds the day's review: the positions opened that session - closed or still open, with where the open ones stood on the close ( `opened_rows()` ) - every closed trade, the mistakes (entered against a flag, exited early, oversized; an entry is judged the day it is taken), the shadow outcomes of plays that were shown but not taken, per-strategy tables, and the session's biggest movers with why they moved and whether the bot traded, offered or missed them. Saved to `daily_reviews` and `data/journal/` , shown on the Reports page.

**The learning storage.** Everything above judges strategies as a whole. To judge one play at a time, the app keeps what every play looked like at the decision next to what happened to it, for three populations: the trades taken ( `trades.entry_context` ), the plays shown and not taken ( `shadow_trades` , from the review) and the replay's simulated trades ( `sim_trades` , from a sink on the runner). One function, `research/features.py play_features()` , describes a play with a fixed, versioned set of keys, and `research/dataset.py` joins the three. The companion guide `docs/AutoTradeBot-learning.pdf` covers the model to be trained on it, its validation and how it plugs into the gates.

**Market regime** ( `engine/market_regime.py` , `quant/regime.py` ). A two-state Markov switching model fitted by EM on three years of SPY daily returns gives the probability the market is in its turbulent state today. It feeds the `turbulent_market` noise flag and the header pill.

## 8. The dashboard and the API

The browser loads `index.html`, which imports `js/main.js`. Each panel is a module with an `init()`; `state.js` holds the shared state and a tiny pub/sub (`on(topic)`, `emit(topic)`); `events.js` opens the WebSocket, and every event either patches the state or triggers a fetch. Nothing is computed in the browser that the engine already knows: the engine's `snapshot()` is the single source of truth.

| Module                                        | Panel / job                                                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| topbar.js, ui.js, tooltips.js                 | the header pills (connection, market session, regime, equity), the drawer and tabs, hover help                     |
| plays.js, chart.js, sheet.js                  | the board of plays, the detail panel with the candle chart and exit routes                                         |
| autopilot.js                                  | Autopilot on/off, dry run, types, the confidence / R:R / loss-stop sliders                                         |
| filters.js, strategies.js, settings.js        | the filter bar (sides, timeframes, sectors), the strategy catalogue with weights in plain words, the scan schedule |
| blotter.js, orders.js, notes.js               | open and closed trades, untracked shares, working orders, Autopilot notes                                          |
| scan.js, watchlist.js                         | the scan status bar and the hot list / buffers / decisions                                                         |
| reports.js, journal.js, movers.js             | the Reports page: reviews, movers, strategy tables                                                                 |
| signals.js, pairs.js, connections.js, quit.js | the Signals page, the pairs desk, the Connections drawer (routing, .env), the quit dialog                          |

### REST routes (server/app.py)

| Route                                                                         | Method    | Engine call                                                                                          |
|-------------------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------|
| /api/state                                                                    | GET       | snapshot(): account, positions, autopilot, filters, market, scan, pnl, venue ...                     |
| /api/plays, /api/plays/{id}/assess   approve   reject   chart                 | GET/POST  | current_plays(), assess_play(), approve_play(), reject_play(), play_chart()                          |
| /api/trades, /api/trades/{id}/close   managed   record, /api/trades/close-all | GET/POST  | trade lists, close_position(), set_trade_managed(), trade_record(), close_all_positions()            |
| /api/positions/untracked/{symbol}/close                                       | POST      | close_untracked()                                                                                    |
| /api/orders, /api/pnl, /api/account/refresh                                   | GET/POST  | active_orders(), pnl, refresh_account_now()                                                          |
| /api/autopilot, /api/filters, /api/strategies[{key}]/reset                    | GET/POST  | set_autopilot(), set_filters(), set_strategy(), reset_strategies()                                   |
| /api/scan, /api/settings, /api/watchlist                                      | GET/POST  | request_scan(kind), set_scan_settings(), watchlist_state()                                           |
| /api/replay[/history], /api/journal[{session}]/review, /api/signals[...]      | GET/POST  | start_replay(), replay_state(), journal_review(), review_session(), signals_state(), check_signals() |
| /api/pairs, /api/pairs/enter, /api/pairs/trades/{id}/close, /api/pairs/chart  | GET/POST  | pairs_state(), enter_pair(), close_pair(), pair_chart()                                              |
| /api/mode, /api/paper/reset, /api/capital[/split]                             | POST      | set_mode(), reset_paper(), set_capital(), set_capital_split()                                        |
| /api/setup/secrets/reconnect/ibkr/test/paper-platform, /api/quit              | GET/POST  | same-machine only: setup_state(), save_secrets(), reconnect(), probe_ibkr(), begin_quit()            |
| /ws                                                                           | WebSocket | a queue on the EventBus; first message is a full snapshot                                            |

### Events on the bus (core/eventbus.py)

`engine.started`, `account.snapshot`, `plays.updated`, `plays.changes`, `play.decided`, `scan.started`, `scan.failed`, `watchlist.updated`, `orders.updated`, `trade.closed`, `trades.removed`, `exit.triggered`, `exit.scaled`, `exit.stop_moved`, `exit.failed`, `exit.not_held`, `autopilot.entered`, `autopilot.would_enter`,

autopilot.skipped, autopilot.blocked, autopilot.daily\_loss, autopilot.config, broker.connected /  
disconnected / reconnected / down / switched, positions.mismatch, position.earnings\_ahead,  
journal.updated, pairs.updated, capital.updated, filters.updated, strategies.updated,  
settings.updated, quit.requested / started / progress / done, engine.disarmed.

## 9. The functions that matter most

If you read only thirty things in the code, read these. Paths are relative to `tos_bot/`.

| File                    | Function(s)                                                                                                                               | What it does                                                                                                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| engine/engine.py        | <code>TradingEngine.start()</code>                                                                                                        | binds the broker, reads the account, launches the seven loops                                                                                                         |
| engine/engine.py        | <code>_due_scan()</code>                                                                                                                  | decides which scan is due (full / gappers / cycle / fast / plays)                                                                                                     |
| engine/engine.py        | <code>_run_scan(kind)</code>                                                                                                              | runs it, records the scan_runs row, replaces the board, notes changes, hands the board to Autopilot                                                                   |
| engine/engine.py        | <code>snapshot()</code>                                                                                                                   | the dict the dashboard renders from; everything else is derived from it                                                                                               |
| engine/engine.py        | <code>assess_play(id)</code>                                                                                                              | session validity, PDT, sizing, arm state, R:R - the yes/no before any order                                                                                           |
| engine/engine.py        | <code>approve_play(id)</code>                                                                                                             | executes a play and books the trade                                                                                                                                   |
| engine/engine.py        | <code>_reconcile_open_trades()</code>                                                                                                     | compares records with the broker's positions; settles a gone position from its fills ( <code>_settle_gone</code> , <code>_exit_fill</code> ); reports size mismatches |
| engine/engine.py        | <code>_refresh_account()</code>                                                                                                           | reads the account; keeps the last snapshot when IBKR is slow (a slow answer never reads as an empty account)                                                          |
| scanner/scanner.py      | <code>Scanner.run_full()</code> / <code>run_cycle()</code> / <code>run_gappers()</code>                                                   | the three scans (section 3)                                                                                                                                           |
| scanner/heat.py         | <code>daily_metrics()</code> ,<br><code>rank_by_daily_heat()</code> ,<br><code>intraday_metrics()</code> ,<br><code>rank_gappers()</code> | how a stock earns attention: relative volume, move in ATRs, distance to the 20-day extreme, range, dollar volume                                                      |
| scanner/watchlist.py    | <code>DayWatchlist.apply_cycle()</code>                                                                                                   | adopt a buffer name into the hot list, keep it waiting, or drop it                                                                                                    |
| scanner/evaluator.py    | <code>evaluate()</code>                                                                                                                   | context, strategies, noise flags, readings, score, signal nudges                                                                                                      |
| strategies/base.py      | <code>StrategyContext</code>                                                                                                              | memoised indicators shared by all setups on one stock                                                                                                                 |
| strategies/base.py      | <code>Strategy._mk_play()</code>                                                                                                          | builds a Play with the geometry floors and the Douglas-style explanation                                                                                              |
| strategies/base.py      | <code>calibrated_probability()</code>                                                                                                     | blends the setup's odds with the pooled record                                                                                                                        |
| scanner/noise.py        | <code>context_flags()</code> , <code>quant_flags()</code> ,<br><code>event_flags()</code>                                                 | the noise checks from the books                                                                                                                                       |
| scanner/filters.py      | <code>expected_r()</code> , <code>rank_score()</code>                                                                                     | a play's value per unit of risk, and its rank                                                                                                                         |
| risk/position_sizing.py | <code>size_play()</code>                                                                                                                  | shares from the risk budget, with every cap and its label                                                                                                             |
| execution/autopilot.py  | <code>AutoPilot.consider()</code> ,<br><code>_pre_gate()</code> , <code>proof_missing()</code> ,<br><code>daily_loss_reason()</code>      | the gates of section 4                                                                                                                                                |
| execution/executor.py   | <code>execute_play()</code> , <code>close_trade()</code> ,<br><code>sync_open_orders()</code> ,<br><code>adopt_working_orders()</code>    | orders in and out, fills booked, orders already at the broker followed after a restart                                                                                |

| File                          | Function(s)                                                                      | What it does                                                                                                                                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| execution/exit_manager.py     | ExitManager.run_once() -> _manage()                                              | stop, target, scale-out, EOD flatten, time stop, break-even, trailing (figure 5)                                                                                                                                                                      |
| execution/protective_stops.py | _protect_positions(), _stand_down(), _watch_stops(), _book_target_fill()         | the stop and the target resting at IBKR for each open trade, in one one-cancels-all group: placed, moved with the record, rebuilt after the scale-out, stood down before the app's own exits, booked when they fill, swept when their trade is gone   |
| execution/order_builder.py    | plan_order(), build_entry_order()                                                | which order type and session a play may use; the limit offset                                                                                                                                                                                         |
| brokers/ibkr_adapter.py       | IbkrBroker (get_account, history_many, place_bracket, get_fills, news_headlines) | everything IBKR, on one asyncio loop thread with timeouts                                                                                                                                                                                             |
| brokers/paper_adapter.py      | PaperBroker.place_order(), poll()                                                | the simulator's fills on real prices                                                                                                                                                                                                                  |
| execution/autopilot.py        | kind_slots(), _kind_full(), settings_changed()                                   | Autopilot's positions and daily entries divide between day and swing trades the way the trading capital's slider does; a settings change hands back the plays it had refused for the day                                                              |
| execution/exit_manager.py     | stop_locked(), intraday_time_stop                                                | a day trade past its setup's window whose stop isn't at break-even is closed (time-stop); the replay's _step does the same                                                                                                                            |
| execution/protective_stops.py | _keep_short_cover(), short_covers()                                              | a broker holding fewer shares than the record for 20 s gets a stop for the shares it holds, resized as they change, the pair once it holds them all                                                                                                   |
| engine/engine.py              | _replay_after_full_scan()                                                        | the morning full scan finishing starts the day's replay (replay.daily): once a session, remembered across a restart                                                                                                                                   |
| persistence/repository.py     | open_trade(), close_trade() - entry_latency_s / exit_latency_s                   | the seconds from an order going out to its fill coming back, kept per trade for the training set and the daily review                                                                                                                                 |
| engine/engine.py              | _settings_changed()                                                              | called by every live setter (split, capital, filters, strategies, Autopilot, account switch, a finished replay): re-sizes and republishes the plays, publishes Autopilot's state, pulls the board re-check forward - and never places an order itself |
| engine/day_state.py           | DayStateOps._save_day(), _restore_day()                                          | saves the board and the scans' state as the day goes; start() picks up the current session's, never a price                                                                                                                                           |
| engine/reconcile.py           | PositionCheck.gone(), share_counts()                                             | when a missing position counts as gone (fresh account, settled connection, grace, two misses)                                                                                                                                                         |
| persistence/repository.py     | open_trade(), reduce_trade(), close_trade(), trade_record(), pnl_summary()       | the trade ledger and its R multiples                                                                                                                                                                                                                  |
| research/in_play.py           | metrics_history(), in_play()                                                     | the day-trade replay's stocks: each past session's hottest by daily heat as of the session before, and its biggest gaps - the morning scan, point in time                                                                                             |

| File                      | Function(s)                                                                                          | What it does                                                                                                       |
|---------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| research/history.py       | IntradayHistory.load(),<br>load_days()                                                               | 5-minute candles kept on disk;<br>load_days fetches only a stock's<br>sessions in play and the four before<br>each |
| research/replay.py        | replay_intraday(),<br>replay_swing(), summarize(),<br>learned_skips()                                | the backtest and its records                                                                                       |
| research/runner.py        | ReplayRunner.start(),<br>replay_job(), session_chunks()                                              | the parallel replay                                                                                                |
| research/weights.py       | evidence_multiplier(),<br>pooled_odds()                                                              | how records change scores and odds                                                                                 |
| research/features.py      | play_features()                                                                                      | a play as one flat, versioned row of<br>features - the same for live, shadow<br>and replayed plays                 |
| research/dataset.py       | training_rows(), write_csv()                                                                         | the training set joined from the three<br>populations                                                              |
| persistence/repository.py | save_sim_trades(),<br>save_shadow_trades()                                                           | the learning tables' writers                                                                                       |
| research/journal.py       | build_review(), opened_rows(),<br>find_mistakes(),<br>shadow_outcomes(), lessons()                   | the daily review: positions opened,<br>trades closed, mistakes, plays not<br>taken, lessons                        |
| pairs/desk.py             | PairDesk.refresh(), watch(),<br>enter(), manage(), close()                                           | the pairs desk: both legs, the z-score<br>bands, the time stop                                                     |
| quant/*                   | adf(), hurst(), half_life(),<br>fit_garch11(),<br>fit_markov_switching(),<br>johansen(), best_band() | the books' models in NumPy                                                                                         |
| util/clock.py             | market_status(),<br>current_session(), time_of_day(),<br>next_trading_day()                          | the NYSE calendar and sessions<br>everything is timed by                                                           |
| server/app.py             | create_app()                                                                                         | routes to engine methods; the<br>WebSocket fan-out                                                                 |

## 10. Trading terms, explained for a computer scientist

| Term                                    | Meaning in this app                                                                                                                                                                                    |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Candle / bar (OHLCV)                    | one time bucket of prices: open, high, low, close and volume. Daily candles for swings, 5-minute candles for day trades. Stored as pandas DataFrames indexed by New York time.                         |
| Long / short                            | long = buy first, profit if the price rises; short = sell borrowed shares first, profit if it falls. Side.sign is +1 / -1 and every formula multiplies by it.                                          |
| Entry, stop, target                     | entry: the price the plan wants; stop: the price at which the idea is wrong and the position is closed; target: where profit is taken. The stop is a sunk-cost guard, not a prediction.                |
| R (risk unit), R multiple               | $R =  \text{entry} - \text{stop} $ per share, the amount risked. A trade that gains 2R made twice what it risked. Expectancy in R is the average R per trade, the one number that compares strategies. |
| Reward-to-risk (R:R)                    | $(\text{target} - \text{entry}) / (\text{entry} - \text{stop})$ . Aziz requires $\geq 2$ ; the ranking uses expected value instead so a tightened stop cannot game it.                                 |
| Position sizing, the 2% rule            | $\text{shares} = (\text{equity} \times \text{risk}\%) / R$ . Risk 1% per trade here (Aziz allows 2%), at most 4% open across positions.                                                                |
| Half-Kelly                              | the Kelly criterion gives the bet fraction that maximises log growth from win rate and payoff; half of it is the conventional safe version. It can only lower the configured risk %.                   |
| Slippage, commission, bps               | the cost of a fill vs the quoted price, and the broker's fee. 1 bp = 0.01%. The replay charges 5 bps slippage and 1 bp commission on every fill.                                                       |
| Bracket order                           | an entry order with a stop order and a target order attached; the broker cancels one when the other fills. IBKR supports it natively.                                                                  |
| Day trade vs swing                      | a day trade opens and closes the same session (and counts against the pattern-day-trader rule in a small live account); a swing is held days.                                                          |
| PDT rule                                | US rule: under \$25k equity, at most 3 day trades in 5 sessions. risk/pdt_guard.py enforces it in live mode.                                                                                           |
| VWAP                                    | volume-weighted average price since the open: the day's fair price for institutions. Being above it is bullish context.                                                                                |
| ATR                                     | average true range: the typical candle range over 14 bars. Used as a volatility yardstick for stops and floors.                                                                                        |
| EMA, RSI, MACD, Bollinger, Keltner, ADX | standard indicators in indicators/ta.py: smoothed price, momentum oscillators, volatility bands, a trend-strength measure.                                                                             |
| Support / resistance                    | price levels where reversals clustered before (analysis/levels.py clusters swing highs and lows).                                                                                                      |
| Gap, gapper, RVOL                       | a gap is today's open far from yesterday's close; a gapper is a stock gapping pre-market on volume; RVOL is today's volume relative to the average - attention.                                        |
| Mean reversion vs momentum              | two regimes of price behaviour. ADF / Hurst / variance-ratio tests (quant/stationarity.py) say which one a series is in; reversal setups are refused on a trending series and vice versa.              |
| Half-life                               | for a mean-reverting series, the time for a deviation to halve (from an AR(1) fit). Sets the expected hold of a reversal trade.                                                                        |
| GARCH(1,1), EWMA                        | volatility models where today's variance depends on yesterday's shock and variance. The forecast sets a stop floor while volatility is rising.                                                         |
| Regime (Markov switching)               | a hidden two-state model (calm / turbulent) fitted to SPY returns; the filtered probability of the turbulent state is the market pill.                                                                 |
| Cointegration, pairs, z-score           | two stocks whose price spread is stationary. The spread's z-score says how stretched it is; enter at the band, exit at the mean, stop past the stop band. Engle-Granger and Johansen are the tests.    |
| Held-out (out of sample)                | the latest third of the replay's sessions, judged separately: a setup must work on data it was not chosen on.                                                                                          |
| Expectancy, profit factor, drawdown     | average R per trade; gross wins / gross losses; the worst peak-to-trough run of losses.                                                                                                                |
| Paper vs live                           | paper = a practice account with real prices and no money (IBKR's, or the built-in simulator). Live = real money on port 4001.                                                                          |

| Term               | Meaning in this app                                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Scale-out          | sell half at the first target and move the stop to break-even; let the rest run to the second target.                               |
| MFE / MAE          | maximum favourable / adverse excursion: the best and worst the trade went while open.                                               |
| Form 4, 8-K, EDGAR | SEC filings: insider trades (Form 4), material events including earnings (8-K item 2.02), and EDGAR is the SEC's free API for them. |

Generated from the code as of 16 September 2026 (master plus the learning storage). When the code changes, the README and this guide should be updated together.